

Lecture 3

Trees, Random Forests, and Boosting

ECON6083: Machine Learning in Economics

Today's Roadmap

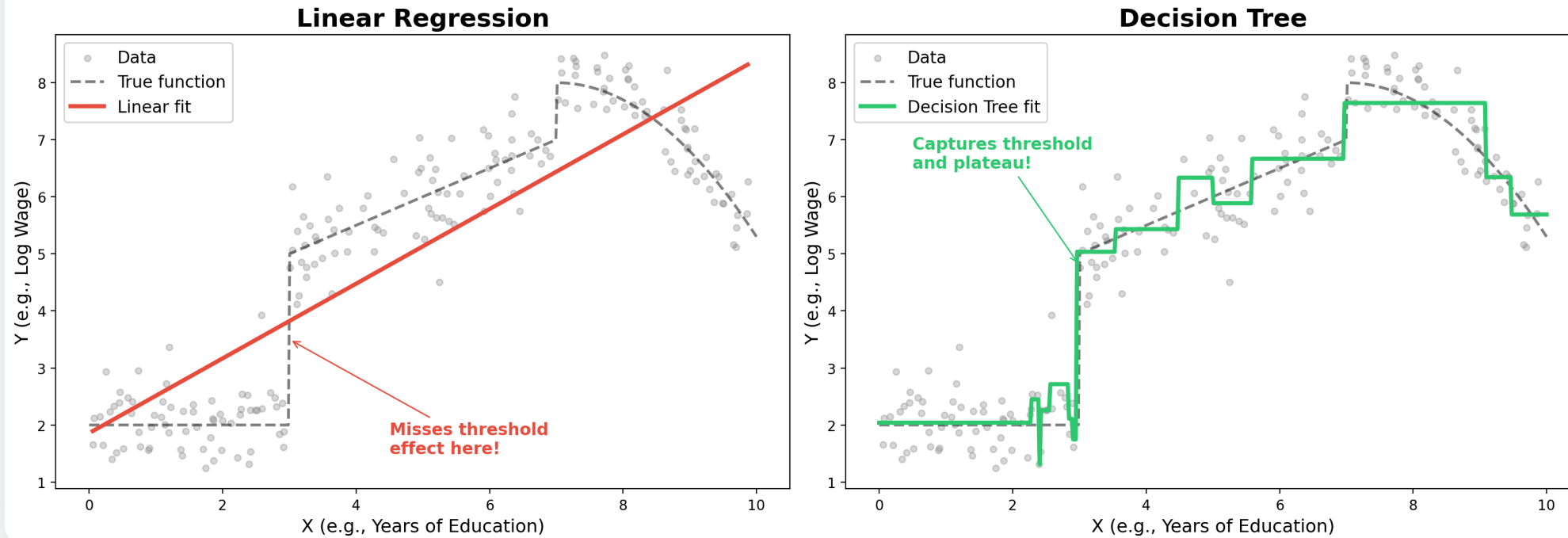
#	TOPIC	FOCUS
1	Beyond Linearity	Threshold effects, interactions
2	Decision Trees	CART, splitting, regularization
3	Random Forests	Bagging, decorrelation, OOB
4	Boosting Methods	Gradient Boosting, XGBoost
5	Interpretability	SHAP, feature importance

Part 1

Beyond Linearity

Linear vs Tree: Capturing Non-Linearity

Why Trees? They Capture Non-Linear Patterns Automatically



The Linear Worldview

Standard linear model:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$$

Key assumptions:

- **Monotonicity:** If $\beta_1 > 0$, increasing X_1 always increases Y
- **Additivity:** Effect of X_1 independent of X_2

Reality: Economic relationships rarely this simple

Threshold Effects

Sheepskin effects in education:

- 11 vs 12 years of schooling (high school diploma)
- Discrete jump in wages beyond linear trend

Credit score cutoffs:

- Score of 619 vs 620
- Bank policy creates discontinuity
- Linear model misses this structural break

Complex Interactions

Interest rate effects on inflation depend on:

- Current unemployment rate
- Corporate debt levels
- International capital flows

The curse of dimensionality:

Specifying all interactions ($X_1X_2X_3\ldots$) in OLS impossible

Linear vs Non-Linear Decision Boundaries

In classification tasks:

- Linear models (logit) draw straight line separators
- Real boundaries often "lumpy" or circular

Example: Credit default

- High income + high debt = risky
- Low income + low debt = safe
- Circular boundary, not linear

Trees vs Linear Models

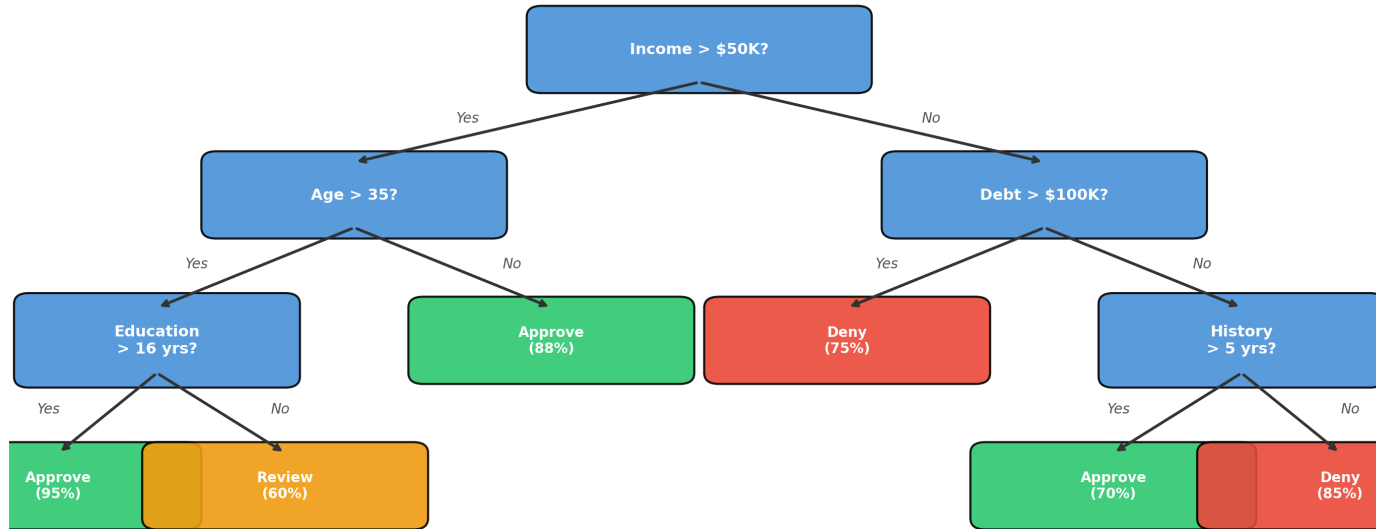
ASPECT	LINEAR MODELS	TREE-BASED METHODS
Functional form	Global linear	Local step functions
Interactions	Manual specification	Automatic discovery
Thresholds	Require dummies	Natural partitioning
Extrapolation	Reliable outside range	Poor outside training range
Interpretability	Coefficients	Visual decision paths
Predictive power	Limited by linearity	Higher with ensembles

Part 2

Decision Trees (CART)

The Tree Intuition

Decision Tree: Credit Approval



Decision tree is a step function:

- Partitions feature space into rectangular regions $R_1, R_2, \dots, R_J$
- **Regression:** Predict mean of observations in region
- **Classification:** Predict mode (most common class)

Greedy Recursive Splitting

At each step: Find best variable X_j and cutoff s to split

"Greedy": Optimize current split only (no lookahead)

- Computationally tractable
- May miss globally optimal tree

"Recursive": Apply same process to each resulting node

Regression Trees: RSS Criterion

Goal: Minimize Residual Sum of Squares

$$\min_{j,s} \left[\sum_{i:x_i \in R_1} (y_i - \bar{y}_{R_1})^2 + \sum_{i:x_i \in R_2} (y_i - \bar{y}_{R_2})^2 \right]$$

Economic example: House prices

- Split 1: Location == 'Downtown'?
- Split 2: SqFt > 2000?
- Split 3: Age < 10 years?

Classification Trees: Impurity Measures

Cannot use RSS for categorical outcomes

Gini Index (CART default):

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

Cross-Entropy:

$$D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}$$

Both minimize when node is pure ($\hat{p} = 0$ or 1)

Economic Example: Recession Prediction

Root node: 50% Recession, 50% Expansion (high impurity)

Split on Yield Curve Inversion:

- **Left (inverted):** 80% Recession → Gini = 0.32
- **Right (normal):** 10% Recession → Gini = 0.18

Result: Split significantly reduces impurity

The Bias-Variance Trade-off

TREE DEPTH	BIAS	VARIANCE	RISK
Deep	Low	High	Overfitting
Shallow	High	Low	Underfitting

Goal: Find optimal depth via regularization

Tree Regularization

METHOD	DESCRIPTION	PURPOSE
<code>max_depth</code>	Limit tree depth	Prevent overfitting
<code>min_samples_leaf</code>	Minimum observations per leaf	Smooth predictions
<code>min_samples_split</code>	Minimum to split a node	Prevent spurious splits
Cost-complexity pruning	Grow full, then prune	Balance fit vs complexity

Recommended: Start with `max_depth=4-6` and `min_samples_leaf=20-50`

Decision Trees in Python

```
from sklearn.tree import DecisionTreeClassifier, DecisionTree ...
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

# --- Classification Tree ---
clf_tree = DecisionTreeClassifier(
    max_depth=4,          # limit depth to avoid overfitting
    min_samples_leaf=20,  # each leaf must have ≥ 20 observations
    criterion='gini',     # 'gini' (default) or 'entropy'
    random_state=42
)
clf_tree.fit(X_train, y_train)
y_pred = clf_tree.predict(X_test)

# --- Visualise the tree ---
fig, ax = plt.subplots(figsize=(16, 8))
plot_tree(clf_tree, feature_names=X_train.columns,
          class_names=['No Default', 'Default'],
          filled=True, rounded=True, ax=ax)
plt.tight_layout(); plt.show()
```

Part 3

Random Forests

The Problem with Single Trees

High variance:

- Small changes in training data
- Completely different tree structure

Economic analogy:

- Single economist: potentially biased
- Survey of 100 economists: consensus more stable

Solution: Ensemble methods

Bagging: Bootstrap Aggregating

Algorithm:

1. Generate B bootstrap samples (sampling with replacement)
2. Train deep tree on each sample: $\hat{f}^{*b}(x)$
3. Average predictions:

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$

Mathematical insight: Averaging B i.i.d. variables with variance σ^2 reduces variance to σ^2/B

The Correlation Problem

If one predictor dominates (e.g., Credit Score):

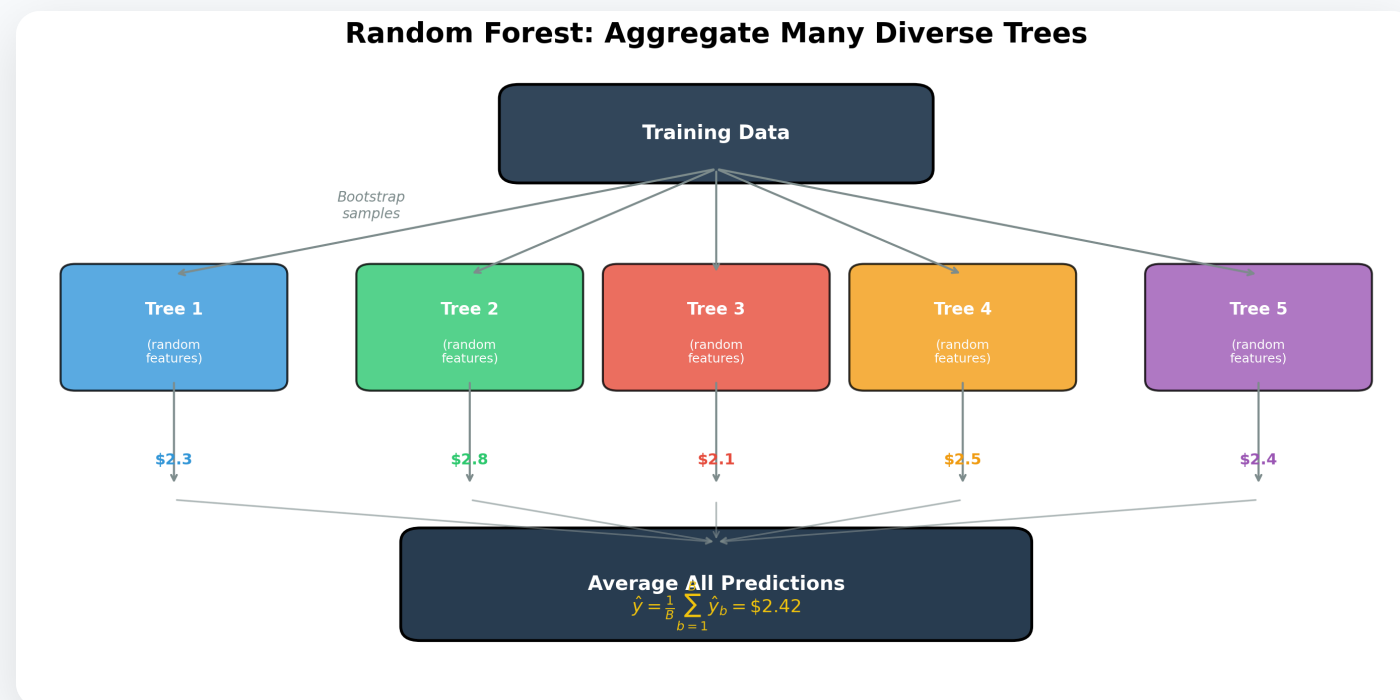
- All bagged trees use it for first split
- Trees highly correlated (ρ high)

Variance of average:

$$\rho\sigma^2 + \frac{1 - \rho}{B}\sigma^2$$

If ρ near 1, bagging doesn't help much!

Random Forest: Decorrelating Trees



The RF innovation: At each split, consider only random subset of m features

Typical choice: $m \approx \sqrt{p}$

Result: Lower correlation ρ , greater variance reduction

Out-of-Bag (OOB) Error

Bootstrap property: Each tree uses $\sim 2/3$ of data, leaves $\sim 1/3$ out

OOB observations:

- Weren't used to train that tree
- Can test the tree's performance

Key benefit: No need for separate validation set! OOB error $\approx$ unbiased estimate of test error

Random Forest: Hyperparameters

PARAMETER	TYPICAL VALUE	EFFECT
<code>n_estimators</code>	100-1000	More trees = better (diminishing returns)
<code>max_features</code>	<code>sqrt</code> or <code>log2</code>	Controls decorrelation
<code>max_depth</code>	None or 10-20	Unlimited depth reduces bias
<code>min_samples_leaf</code>	1-10	Larger = smoother predictions

Computation: Trees grown in parallel (fast on multi-core)

Random Forests in Python

```
from sklearn.ensemble import RandomForestRegressor, RandomFo ...

# --- Fit Random Forest ---
rf = RandomForestRegressor(
    n_estimators=500,      # number of trees
    max_depth=10,         # limit depth per tree
    max_features='sqrt',  # features per split for decorrelation
    min_samples_leaf=10,
    oob_score=True,       # enable out-of-bag error estimate
    n_jobs=-1,            # use all CPU cores
    random_state=42
)
rf.fit(X_train, y_train)

# --- Evaluation ---
print(f"OOB R²: {rf.oob_score_:.3f}")

# Feature importance
for name, imp in sorted(zip(X_train.columns, rf.feature_impo ...
                        key=lambda x: -x[1]):
    print(f" {name:20s} {imp:.4f}")
```

Part 4

Boosting Methods

Bagging vs Boosting

ASPECT	BAGGING (RANDOM FOREST)	BOOSTING (XGBOOST)
Training	Parallel, independent	Sequential, adaptive
Learners	Deep trees (strong)	Shallow trees (weak)
Focus	Reduces variance	Reduces bias
Mechanism	Averaging predictions	Learning from residuals
Risk	Less overfitting	Can overfit without regularization
Best for	High variance problems	High bias problems

Gradient Boosting

Sequential refinement:

1. Start: $F_0(x) = \bar{y}$ (predict mean)
2. Calculate residuals (pseudo-residuals): $r_i = y_i - F_{m-1}(x_i)$
3. Fit shallow tree to residuals: $h_m(x)$
4. Update: $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$
5. Repeat

η = learning rate (shrinkage), typically 0.01-0.1

Boosting: Economic Analogy

Wage prediction example:

Step 1: Education explains 60% of variance

Step 2: Residuals large for tech workers → Next model learns Industry matters

Step 3: Add correction, residuals large for older workers → Next model learns Age matters

Sequential refinement of errors

Why Weak Learners?

Shallow trees (depth 3-6):

- Each captures simple pattern
- Won't overfit on their own
- Stable building blocks

Learning rate η :

- Small values (0.01) = slow learning
- More iterations needed
- Better generalization

Trade-off: Speed vs accuracy

Bagging: The Variance Decomposition

Why averaging trees reduces error:

For B trees with correlation ρ and variance σ^2 :

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B T_b \right) = \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2$$

As $B \rightarrow \infty$: Variance $\rightarrow \rho \sigma^2$ (not zero!)

Key insight: Bagging helps only if $\rho < 1 \rightarrow$ need **decorrelated** trees

Random Forest achieves this via:

1. Bootstrap sampling (different training sets)
2. Random feature subsets at each split ($m \approx \sqrt{p}$)

Gradient Boosting: Loss Function View

General framework: Minimize empirical risk

$$F_M(x) = \sum_{m=1}^M \eta \cdot h_m(x), \quad h_m = \arg \min_h \sum_{i=1}^N \underbrace{\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)}}_{\text{negative gradient}} \cdot h(x_i)$$

For squared loss: Negative gradient = residual $y_i - F_{m-1}(x_i) \rightarrow$ standard gradient boosting

For logistic loss: Negative gradient = $y_i - p_i$ (pseudo-residual) $\rightarrow$ Gradient Boosted Trees for classification

XGBoost vs LightGBM

	XGBOOST	LIGHTGBM
Tree growth	Level-wise (wide)	Leaf-wise (deep)
Split finding	Exact + approximate	Histogram-based
Speed	Fast	Faster (2-10x)
Memory	Higher	Lower
Best for	Medium data, accuracy priority	Large data, speed priority

Recommendation: Both excellent. Start with XGBoost (more robust defaults); switch to LightGBM for $n > 10^6$.

Modern Boosting: XGBoost

Advantages over standard Gradient Boosting:

- Optimized for speed (parallelization, GPU support)
- Handle missing data automatically
- Built-in regularization (L1/L2 penalties)
- Built-in cross-validation and early stopping
- Feature importance scores

State-of-the-art for tabular economic data

XGBoost in Python

```
import xgboost as xgb
from sklearn.ensemble import GradientBoostingRegressor

# --- Gradient Boosting (sklearn) ---
gb = GradientBoostingRegressor(
    n_estimators=500,
    learning_rate=0.05,      # shrinkage
    max_depth=4,             # shallow trees
    subsample=0.8,           # stochastic GB
    n_iter_no_change=20,     # early stopping
    random_state=42
)
gb.fit(X_train, y_train)

# --- XGBoost (recommended) ---
xgb_model = xgb.XGBRegressor(
    n_estimators=1000,
    learning_rate=0.05,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=0.8,    # feature subsampling
    reg_alpha=0.1,          # L1 regularization
    reg_lambda=1.0,         # L2 regularization
    early_stopping_rounds=30,
    random_state=42
)
xgb_model.fit(X_train, y_train, eval_set=[(X_test, y_test)], ...)

print(f"Test R²: {xgb_model.score(X_test, y_test):.3f}")
xgb.plot_importance(xgb_model, max_num_features=10)
```

Part 5

Interpretability & Conclusion

The Interpretability Trade-off

MODEL	STRENGTH	WEAKNESS
Logistic Regression	Clear equation, odds ratios	Linear only
Decision Tree	Visual path, rule-based	Unstable, weak predictions
Random Forest	Feature importance, robust	No closed form
XGBoost	Best predictions, importance	"Black box"

Solution: Feature importance + SHAP values for any model

Feature Importance Methods

METHOD	HOW IT WORKS	PROS	CONS
Impurity-based	Sum of impurity reductions across splits	Fast, built-in	Biased to high-cardinality features
Permutation	Shuffle feature, measure perf. drop	More reliable, model-agnostic	Slower, requires validation set
SHAP	Game-theoretic marginal contributions	Local + global explanations, theoretically sound	Computationally intensive

SHAP Values

SHAP (SHapley Additive exPlanations):

- Decomposes individual predictions into feature contributions
- "Why did model predict this person would default?"
- Theoretically grounded (game theory Shapley values)
- Compatible with any model (model-agnostic)

Use cases:

- Explain individual loan decisions to customers
- Audit model fairness across demographic groups
- Understand driver variables in forecasting

Practical Workflow

Step 1: Baseline

- Start with train/test split and simple linear model
- Establish performance benchmark

Step 2: Tree Ensemble

- Fit Random Forest (robust, less tuning)
- Fit XGBoost (higher accuracy, more tuning)

Step 3: Validation

- Cross-validation for hyperparameters
- OOB error for Random Forest
- Early stopping for XGBoost

Step 4: Interpretation

- Feature importance plots
- SHAP summary plots
- Partial dependence plots

Key Takeaways

1. **Trees capture non-linearity** — thresholds and interactions automatically
2. **Always use ensembles** — Random Forest or XGBoost, not single trees
3. **Tune hyperparameters** — `max_depth`, `learning_rate`, `min_samples_leaf`
4. **Use OOB or early stopping** — built-in validation, no separate holdout needed
5. **Explain predictions** — feature importance and SHAP for transparency
6. **Choose RF vs XGBoost** — RF for robustness, XGBoost for accuracy

Common Pitfalls

PITFALL	WHY IT MATTERS	SOLUTION
Using single trees	High variance, unstable predictions	Always use RF/XGBoost
Ignoring tuning	Suboptimal performance, overfitting	Cross-validation, early stopping
Black-box predictions	No stakeholder trust, regulatory issues	SHAP values, partial dependence
Causal inference	Adaptive trees lead to biased estimates	Use honest estimation (causal forests)

Further Reading

PAPER	AUTHORS	KEY CONTRIBUTION
<i>Random Forests</i>	Breiman (2001), Machine Learning	Original RF paper, OOB error
<i>XGBoost: A Scalable Tree Boosting System</i>	Chen & Guestrin (2016), KDD	XGBoost algorithm & optimizations
<i>Consistent Individualized Feature Attribution</i>	Lundberg & Lee (2017), NeurIPS	SHAP values theory
<i>Machine Learning: An Applied Econometric Approach</i>	Mullainathan & Spiess (2017), JEP	ML perspective for economists

Data and Code

This lecture:

- Slides source: `_slides/Lecture03-Trees, Random Forests, and Boosting-Slides.md`
- Exercises: `exercises/Lecture03-Exercise.ipynb`

Key packages:

- Python: `scikit-learn`, `xgboost`, `lightgbm`

Datasets:

- California housing; Kaggle competitions

Thank You!

Next Lecture: Cross-Validation and Model Selection

Reading: Arlot & Celisse (2010), "A Survey of Cross-Validation"